

根据行车轨迹估计红绿灯周期问题

摘要

在现代社会，我们的出行因众多地图服务的普及而变得更加便捷。一家电子地图服务提供商决定通过分析和整理大量用户的行车轨迹数据，来估计这些信号灯的周期，以应对许多信号灯尚未联网这一挑战。

问题一中，在交通信号灯周期不变的情况下，我们求取了附件 1 的 5 个红绿灯的周期。首先进行数据预处理，没有发现异常值和缺失值。我们定义在两个时间点内位置未发生变化车辆处于停止状态。然后我们求出每次的停车时长，设置一个阈值，将停车频率过高的部分剔除，除去因非红灯因素，如交通堵塞等引发的停车情况。利用傅里叶变换找到时间序列中的主要频率，估计交通信号灯的大致周期。最终得到附件 1 中路口 A1、A2、A3、A4、A5 的红灯时长分别为：48s、29s、48s、38s、45s，绿灯时长分别为：58s、59s、58s、50s、43s。

问题二中，解决了什么问题；应用了什么方法；得到了什么结果。

问题三中，解决了什么问题；应用了什么方法；得到了什么结果。

结尾段：可以总结下全文，也可以介绍下你的论文的亮点，也可以对类似的问题进行适当的推广。

关键词：傅里叶变换 交通信号灯周期 3 4 5

一、问题背景与重述

1.1 问题背景

交通是社会发展与国民经济的基础性产业，与人民群众的生活、生产和国民经济的发展息息相关。如今，在各种地图服务下，我们的出行变得越来越方便。某电子地图服务商通过获取交通信号灯的周期，为司机提供了良好的导航服务。为解决很多信号灯未接入网络的问题，该公司决定通过分析并总结大量客户的行车轨迹数据，估计出交通信号灯的周期。

1.2 问题重述

1. 在信号灯周期固定不变的条件下，根据附件 1 数据，求出附件 1 中 5 个路口的信号灯周期，并将结果填入表 1。

2. 讨论样本车辆比例、车流量、定位误差等因素对上述模型估计精度的影响，对模型做出调整后求出附件 2 中另外 5 个路口的相应方向的信号灯周期，将结果填入表 2。

3. 在信号灯周期有可能发生变化的条件下，判断附件 3 中 6 个关路口相应方向的信号灯周期是否有变化，求出周期变化的时刻，新旧周期参数，将结果填入表 3，并指出识别周期变化所需的时间和条件。

4. 根据附件 4 某路口样本车辆的轨迹数据，求出该路口信号灯的周期。

二、模型假设与符号说明

2.1 模型假设

1. 假设所有信号灯只有红、绿两种状态，不考虑黄灯状态。
2. 假设第一问所有车都可看作一个质点，不考虑车辆比例，车辆形状的影响。
3. 假设所给的附件数据都是真实的，有效的。
- 4.

2.2 符号说明

表 1 Example 1

符号	说明	单位
Δt_i	每次停车的持续时长	秒 (s)
T	红绿灯周期	秒 (s)

三、模型的建立与求解

3.1 问题一模型的建立与求解

3.1.1 问题一分析

在解决第一问时，需要考虑车辆因排队而在红灯处停留的时间。因为车辆停留的最长时间不仅可能反映了单个红灯的时长，还可能包括由于交通拥堵或者多个红灯周期导致的连续停留。这些因素需要通过以下方式考虑：

车流量分析：分析不同车辆在同一信号灯下的停留时间可以帮助我们区分是否是因为红灯导致的停止。如果多辆车在大致相同的时间点频繁开始和结束停留，这可能表明它们是由信号灯控制，而不仅仅是交通堵塞。

位置分析：通过观察停车的位置，可以确定是否所有车辆都在同一地点停止（例如在交叉口前），这有助于确认停车是否与信号灯相关。A1 至 A5 轨迹图见附录三。

时间窗口：分析特定时间窗口内的车辆行为，可以帮助识别出由于信号灯导致的停车模式，从而更准确地估计红灯和绿灯的时长。

我们可以进一步分析各个车辆在相同或相似位置的停留时间，以及这些时间的重叠程度，来判断这些停留是否与信号灯周期同步。

3.1.2 问题一模型的建立与求解

一. 数据预处理：

我们对附件一中的 A1 至 A5 数据进行了异常值分析和缺失值检测，结果显示没有任何坐标点（ x 或 y ）的数据超过均值三倍标准差的异常范围，同时也未检出缺失值。所有交叉口的数据均未标识出需要特别注意的潜在异常值。这表明数据的质量可能较高，或者数据之间的变异性较小。

二. 模型建立与求解：

模型的求解过程如图 1 所示：

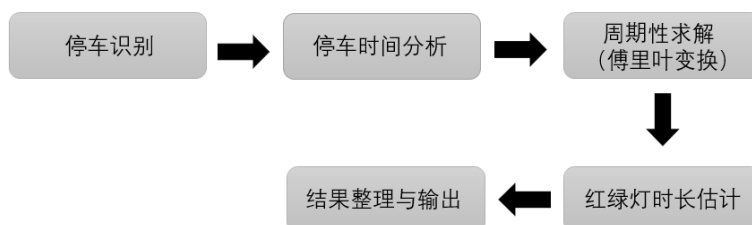


图 1

1. 停车识别：

我们定义若当前位置和下一个位置相同，则判定车辆停止（即 x 和 y 坐标在连续两个时间点内没有变化），并使用这一定义在数据中识别出车辆的停车事件。

2. 停车时间分析:

我们选取出连续的停止时间区间 $[t_s, t_e]$, 计算出每次停车的持续时长 Δt_i , 其求解方法为:

$$\Delta t_i = t_s - t_e \quad (1)$$

其中, t_s 是车辆开始停止的时间, t_e 是车辆停止结束时间。

3. 周期性求解:

本次的周期求解我们采用了傅里叶变换方法求解。

傅里叶变换^[1] 的核心作用是将时间域中的信号转换到频率域中。这种转换使得我们可以从不同的角度理解同一个事物, 有助于简化那些在时域中难以解决的问题, 因为在频率域中, 这些问题往往能够更加简单地得到处理。

为了分析停车时间的周期性, 我们创建了一个信号数组, 其中 1 表示车辆停止, 0 表示无停止。表示为:

$$S_t = \begin{cases} 1 & , P_t > a \\ 0 & , otherwise \end{cases} \quad (2)$$

S_t 表示 t 时刻状态, P_t 表示 t 时刻速度。

我们根据 A1 至 A5 路口的信号灯周期数据, 进行可视化, 具体见附录二。以下是 A1 路口信号灯周期的具体情况及可视化结果: 路口交通信号随时间的变化。在图中, 1 表示车辆停止 (红灯), 而 0 表示车辆通过 (绿灯)。

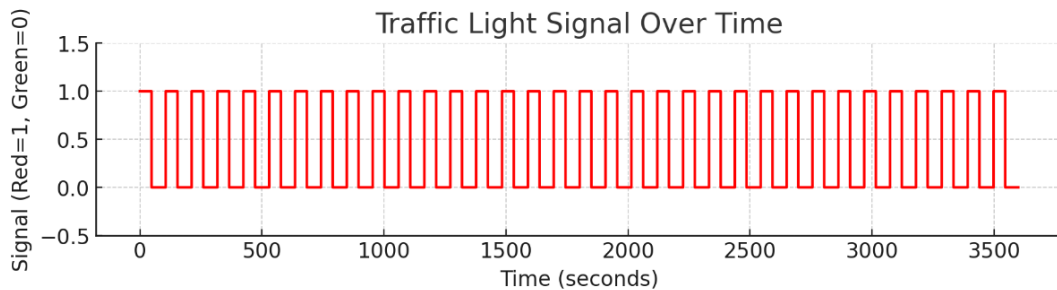


图 2 A1 路口

在上述公式 (2) 中, a 代表速度阈值 (此处取 0.5), 使用该阈值以确定哪些时间段的停车是由于交通信号控制 (比如红灯) 导致的, 而非其他原因如交通堵塞或车辆自行停车等。以此确定哪些时段更可能是由交通信号控制的停车。

阈值的具体应用如下:

(1) 首先, 利用先前估算的交通信号周期, 将信号数组划分为若干段, 每个段的长度与一个信号周期相等。

(2) 对于每一个周期段, 求出该段内的停车标记 (1 代表停车, 0 代表行驶) 的平均值, 以此来得出该段的停车频率。

(3) 将计算出的每个周期段的停车比例与阈值 0.5 进行比较:

若某个周期段的停车频率达到或超过 0.5，则表明该周期内大多数时间车辆处于停止状态，这通常是因为红灯导致的。

若停车频率低于 0.5，则表示在该周期内，车辆的停止时间少于行驶时间，这种停车更可能是偶发性的或由其他因素引起的。

我们将傅里叶变换应用到信号上，从频域中识别出停车时间的主要周期性成分，从而估算出红灯和绿灯的周期。

对每次停车持续时间 Δt_i 使用傅里叶变换，找到时间序列中主要频率（即停车时间的重复频率）的成分，以此估计交通信号灯的大致周期：

$$F(\omega) = \int \Delta t_s(t) e^{-i\omega t} dt \quad (3)$$

其中， ω 表示频率， t 表示时间， $e^{-i\omega t}$ 为复变函数。

周期 T 求解：

$$T = \frac{2\pi}{\omega} \quad (4)$$

我们将傅里叶变换结果可视化，如图 3 是 A1 路口的可视化结果：

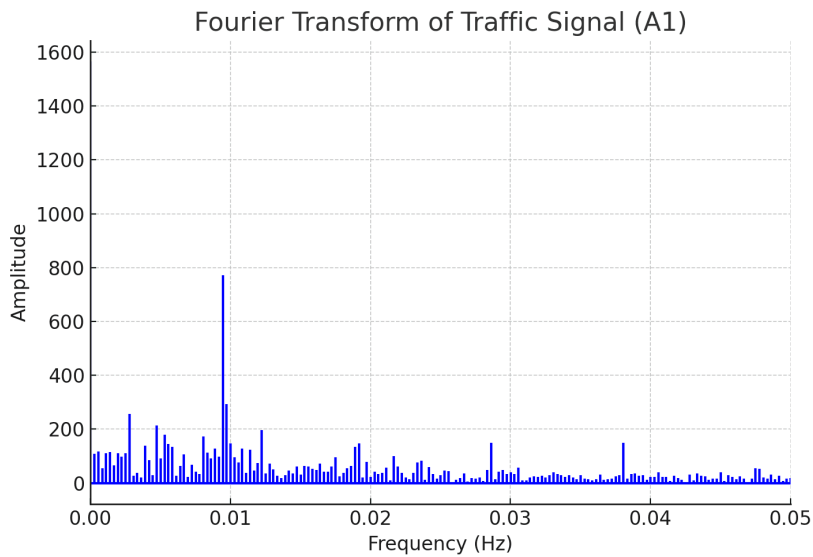


图 3

这个图表显示了信号中不同频率的振幅。较高的振幅对应于交通信号中更显著的周期性成分，其中最突出的峰值对应于主要的红绿灯循环周期。

4. 信号灯状态表示：

假设信号灯状态 H_t 为周期函数，周期为 T ，可表示为：

$$H_t = \begin{cases} red & , \sin\left(\frac{2\pi t}{T}\right) > 0 \\ green & , otherwise \end{cases}$$

5. 红绿灯时长估计：

为了区分交通拥堵和红灯引起的停车，我们进一步分析了停车频率和持续时间的周期性，确保分析反映的是信号灯的周期，而非交通拥堵。根据周期性分析的结果，我们计算了红灯和绿灯的具体时长。

6. 结果整理与输出：

最后，我们为每个路口提供了红灯和绿灯的时长估计，以及整个信号周期的长度。结果如下：

路口	A1	A2	A3	A4	A5
红灯时长（秒）	48	29	48	38	45
绿灯时长（秒）	58	59	58	50	43

3.2 问题二模型的建立与求解

3.2.1 问题二分析

问题二让我们分析各种因素对第一问根据轨迹估计红绿灯周期精度的影响。我们主要考虑定位误差、样本车辆比例、车流量三个方面。

1. **样本车辆比例**：当选取的车辆样本较少时，采集到的数据无法全面覆盖各个时间点或不同的交通状况，从而很难准确捕捉到信号灯的 actual 运作模式。这种数据的不完整性可能会导致对信号灯状态分析的误差，所得到的周期估计可能会出现较大的波动，无法稳定地反映交通流的真实特性。

2. **车流量**：在交通流量较高的情况下，数据的连贯性通常会更好。更多的车辆通过同一交叉口时，每辆车都有可能成为数据采集点，从而积累更多关于信号灯周期的信息。而当车流量较低时，通过路口的车辆间隔变长，这可能导致在某些时间段内无法捕获到信号灯状态的变化。

3. **定位误差**：定位误差是车辆实际位置与定位系统记录位置的偏差，对交通管理系统的判断至关重要。如果定位误差较大，它可能导致对信号灯状态变化的错误判断。例如，系统可能错误地认为车辆已经在红灯时停止，而实际上车辆可能因为定位误差显示在停车线之前或之后。同样，系统可能判断车辆在绿灯亮起时并未及时移动，而实际上车辆已经根据交通信号开始行驶。我们引入高斯噪声，计算定位误差影响下改进的速度。

3.2.2 问题二模型的建立与求解

一. 模型改进及附件二信号灯周期的求解：

将附件二数据带入问题一求解周期的模型，结果如下：

B1 路口：红灯持续 20 秒，绿灯持续 86 秒，信号周期约 106 秒。

B2 路口：红灯持续 44 秒，绿灯持续 72 秒，信号周期约 116 秒。

B3 路口：红灯持续 80 秒，绿灯持续 1120 秒，信号周期 1200 秒。

B4 路口：红灯持续 15 秒，绿灯持续 91 秒，信号周期约 106 秒。

B5 路口：红灯持续 35 秒，绿灯持续 81 秒，信号周期约 116 秒。

观察得知，B3 路口绿灯持续时间为 1120 秒，显然偏离现实生活的正常数据。这可能表明该路口有异常数据或极不寻常的交通信号设置。

分析傅里叶变换结果，对原有模型进行改进。

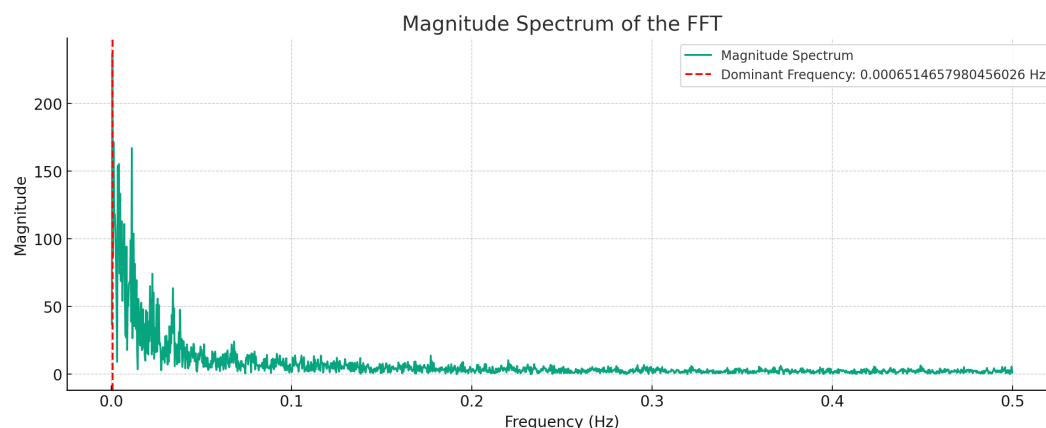


图 4

从傅里叶变换的幅度谱中（图 4），我们发现主导频率约为 0.000651Hz，这对应于一个周期约为 1535 秒。这个周期明显长于通常的交通信号周期，可能表明信号中包含了非周期性因素或数据中的噪声较多：

（1）周期过长：主导周期远超常见的交通信号周期，这可能是由于数据中的非周期性停车事件或噪声导致的。

（2）周期估计的准确性：如果交通流中断或不规则，或者停车数据中包含太多非信号引导的停车（如随机停车），这可能导致周期估计失真。

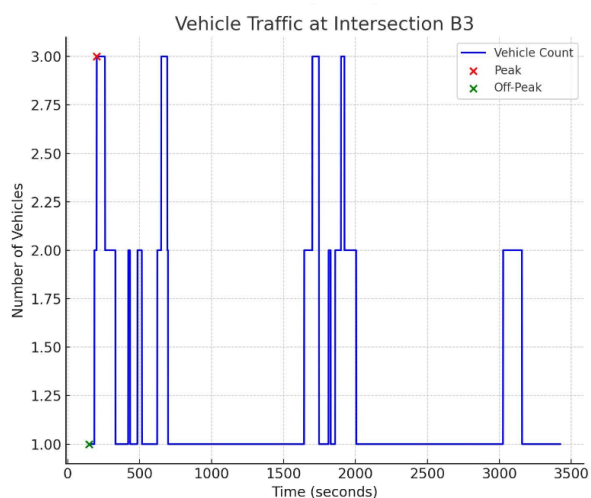


图 5 B3 峰值可视化

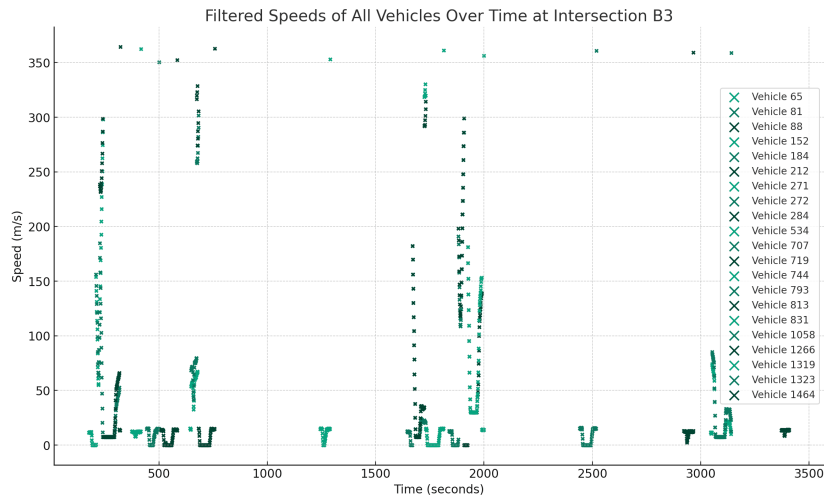


图 6 B3 速度可视化

由上图图 5B3 峰值图、图 6 可知并结合附件二可知，B3 路口数据相对于其他路口数据，样本量较小，车辆比例小。由高峰图可知，它的数据较为集中，主要集中在两个峰。

由此，我们决定对原有模型进行改进，以便求取可以求取如 B3 这类特殊数据的信号灯周期。

我们尝试了选取特定的时间窗口的办法，缩短周期，减少异常周期的影响，并为每个窗口根据车流量赋予权重，车流量大的权重高，反之，车流量较少的权重低。从而降低稀疏车流量对整个周期计算的影响。但结果并不理想。

于是我们采用 DBSCAN 聚类算法进行红绿灯周期计算，优点是不需要考虑时间窗口的选取，即不考虑时间窗口特定选取带来的误差。我们在求取周期显著有效的范围，由图 6 分析可得 250 到 750 秒之间和 1600 到 2000 秒之间数据点较为集中，用这两个范围内求取的周期较为准确，我们在这两个范围内进行聚类，以尝试识别由红绿灯控制的停车点，计算出合理的信号灯周期。

具体求解步骤如下：

1. 数据加载与预处理

首先，我们加载了提供的车辆轨迹数据，并筛选出了特定时间段（time 在 250 到 750 秒之间）的数据，以聚焦于一定时间窗口内的车辆行为。

2. 速度计算与停车时段识别

速度计算：通过计算车辆位置的变化（基于 X 和 Y 坐标），我们得到了每辆车每秒的速度。

停车时段识别：根据速度计算结果，我们认为当车速小于 1 米/秒时车辆处于停止状态。我们进一步识别出持续时间超过 10 秒的停车时段，以排除短暂的停车情况，如暂停或交通拥堵引起的短暂停止。

速度计算方法如下：

$$V_t = \frac{\sqrt{(x_t - x_{t-1})^2 - (y_t - y_{t-1})^2}}{\Delta t} \quad (5)$$

其中， V_t 是 t 时刻的车辆速度。

3. DBSCAN 聚类分析

使用 DBSCAN 聚类算法，我们根据停车位置对停车事件进行了聚类，以尝试识别由红绿灯控制的停车点。这帮助我们确定可能的信号灯位置和相关的停车模式。

4. 红灯时间分析

每个聚类中的停车时长被视为红灯时长的估计值。我们计算了每个聚类内停车时长的平均值，以得出每个信号灯控制点的平均红灯时间。

5. 绿灯时间分析

初步绿灯时间计算：我们尝试通过计算两次停车之间的时间来估计绿灯时长。初步的方法是从连续两次红灯开始之间的时间间隔中减去红灯时长。改进绿灯时间计算：观察到初步方法可能因为数据重叠或错误标记而导致不准确的结果，我们对方法进行了调整。我们使用了每次红灯结束到下一次红灯开始之间的非停车时间作为绿灯持续时间的更合理估计。

经过改进后，计算结果如下：

B1 路口：红灯持续 45 秒，绿灯持续 22 秒，信号周期约 67 秒。

B2 路口：红灯持续 51 秒，绿灯持续 24 秒，信号周期约 75 秒。

B3 路口：红灯持续 45 秒，绿灯持续 41 秒，信号周期约 86 秒。

B4 路口：红灯持续 43 秒，绿灯持续 65 秒，信号周期约 108 秒。

B5 路口：红灯持续 41 秒，绿灯持续 28 秒，信号周期约 69 秒。

由上述结果可知，对于 B3 这类样本量少，数据集中在路口的路口信号灯周期求取，我们采取 DBSCAN 聚类算法算出的结果较好，但对于其他类型数据的路口，采用 DBSCAN 聚类算法算出的结果不够准确。而傅里叶变换（FFT）适用于车辆停车数据表现出明显的时间间隔周期性的情况，如在特定时间周期内车辆重复停车和启动。我们将分析各个路口的数据，找到适合求取它信号灯周期的算法。

分析下图图 7（B1 至 B5 路口高峰图），我们得出以下判断：

B1、B3、B4、B5：这些数据集的车辆路径图显示了显著的转弯或停车点，这表明车辆在这些点停留时间较长，可能是红灯引起的。我们使用 DBSCAN 聚类，以识别停车模式和估计红灯时长。

B2：这些数据集显示了车辆经常沿直线移动，停车次数看起来较少。对于这些数据集，如果时间序列的高峰值确实与红灯周期相关，FFT 可以帮助识别红绿灯周期。我们采用 FFT 分析，以确定红绿灯周期。

其他优化模型的方法：

对于样本比例小，且没有明显高峰期的数据集进行机械生成性对抗，模拟生成新的数据点，拓宽样本数量，再结合数据平滑，如移动平均或高斯平滑，来降低噪声，

提取出更加明显的周期特征。

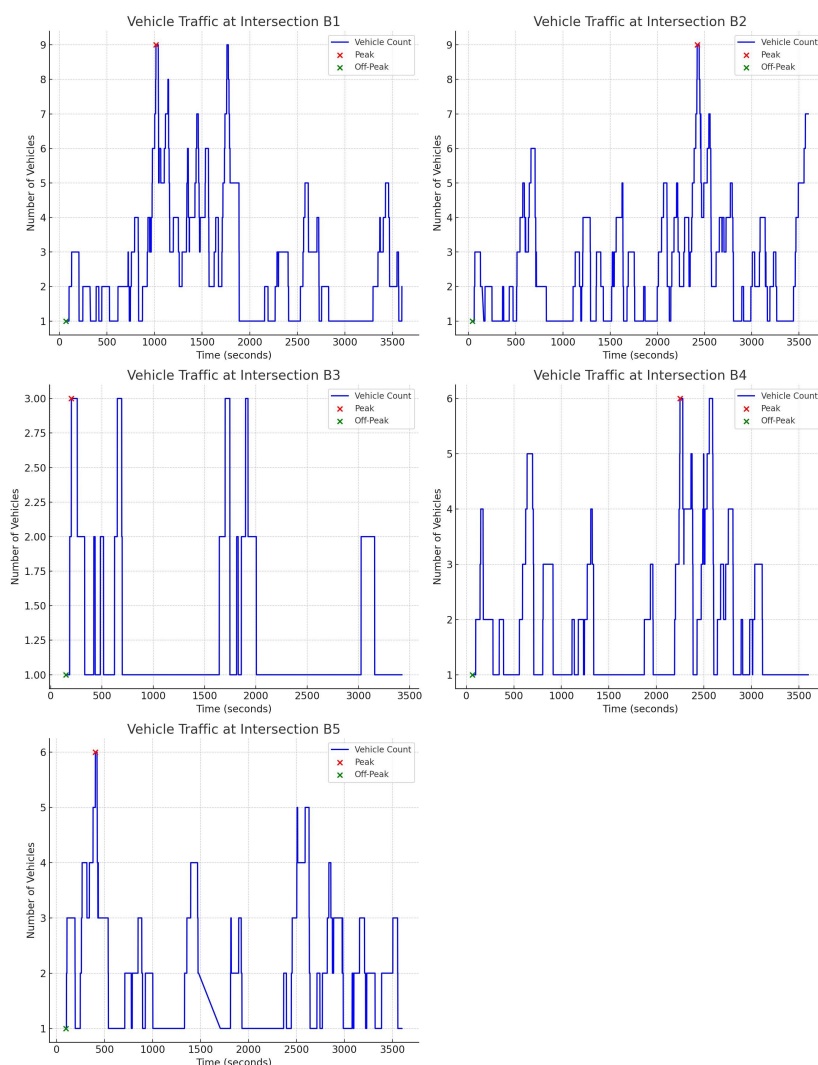


图 7 B1 至 B5 路口高峰图

最终，我们得出最佳的 B1 至 B5 路口信号灯周期如下：

路口	B1	B2	B3	B4	B5
红灯时长（秒）	45	44	45	43	41
绿灯时长（秒）	22	72	41	65	28

二. 各类因素影响分析：

1. 样本车辆比例和车流量：

由上述（一. 模型改进及附件二信号灯周期的求解）过程可知，对 B3 类这类样本量少（即样本车辆比例较小）数据集中（高峰期车流量大）的路口数据集，对问题一中使用的傅里叶变换（FFT）求解，计算出的周期显著大于正常值。故而知车辆比例和车流量对原算法的影响较大。

2. 定位误差:

对于定位误差,我们适当改变原有的 x,y 数据,模拟误差来模拟现实生活中可能出现的由于定位错误所造成的误差,模拟数据自然波动。分别求出在各类改变下的周期,对比原有数据,判断出影响效果,此处我们将原有的 x,y 坐标值浮动 5%。

我们通过使用上述改进模型计算红绿灯周期。具体结果如下图:

路口	B1	B2	B3	B4	B5
原数据红灯时长(秒)	45	44	45	43	41
原数据绿灯时长(秒)	22	72	45	65	28
浮动后红灯时长(秒)	42	42	40	42	41
浮动后绿灯时长(秒)	19	25	151	47	38

3.3 问题三模型的建立与求解

对于如何进行信号灯周期变化的判断,我们选择 CUSUM 算法(累积和控制图)来解决这一问题。

CUSUM 方法是一种序列化检测技术,适用于监测和识别过程中偏离标准或期望行为的时间点。以下是实现 CUSUM 方法的具体步骤:

1. 确定目标值和阈值:

目标值是指在任何给定过程或系统中期望达到的性能标准。以交通信号灯系统为例,目标值可能是平均红灯时长,即城市交通管理中期望的平均等待时间。

阈值则是用于评估系统性能是否发生了重要变化的界限。确定阈值需要仔细分析历史数据和具体需求,以确保系统的响应敏感度既不过度敏感也不迟钝。

2. 计算 CUSUM 分数:

首先选择一个起始点,这个起始点可以是数据集的平均值或中位数。对于接下来的每一个新的数据点,计算它与目标值的偏差,并将这个偏差累加到之前的 CUSUM 分数上。此外,为了能够检测正向和负向的变化,分别计算向上和向下的 CUSUM 分数。这样可以更全面地捕捉到任何方向上的系统变化。

3. 分析 CUSUM 分数:

监控 CUSUM 分数的变化,特别是持续的上升或下降,这可能表明过程中发生了变化。当 CUSUM 分数超过预设的阈值时,可以认为是发现了一个变化点。

4. 采取行动:

首先,我们要求出有效的停车持续时间 Δt_i ,我们规定 $\Delta t > 10s$ 为有效停车持续时间。

我们一旦检测到变化点，分析变化前后的数据，以评估变化的性质和影响。可能需要调整信号灯控制策略或进行进一步的调查。

3.3.1 模型的建立与求解

3.3.2 模型的优缺点

3.4 问题四模型的建立与求解

四、模型评价与推广

4.1 模型的优缺点

4.2 模型的改进与推广

五、参考文献

[1] lzzdfg, 傅里叶变换概念及公式推导, <https://blog.csdn.net/lzzdf>, 2024-04-20

[2] MingChaoSun, 高斯噪声, <https://blog.csdn.net/sunmc1204953974/article/details/50623071>, 2024.

[3] 许嘉, 韦婷婷, 于戈, 黄欣悦, 吕品. 题目难度评估方法研究综述 [A].

六、附录

某一问题的详细证明或求解过程；

自己在网上找到的数据；

比较大的流程图；

较繁杂的图表或计算结果

A附录一：问题一代码

```
1 import pandas as pd
2 import numpy as np
3 from scipy.fft import fft, fftfreq
4
5 def detect_stop_periods(data):
```

```

6 data_sorted = data.sort_values(by=['vehicle_id', 'time'])
7 data_sorted['x_shifted'] = data_sorted.groupby('vehicle_id')['x'].shift(-1)
8 data_sorted['y_shifted'] = data_sorted.groupby('vehicle_id')['y'].shift(-1)
9 data_sorted['stopped'] = ((data_sorted['x'] == data_sorted['x_shifted']) &
10 (data_sorted['y'] == data_sorted['y_shifted']))
11 data_sorted['stopped'] = data_sorted['stopped'].fillna(False)
12 data_sorted['stop_begin'] = data_sorted['stopped'] & (~data_sorted['stopped'].shift(1).fillna(False))
13 data_sorted['stop_end'] = data_sorted['stopped'] & (~data_sorted['stopped'].shift(-1).fillna(False))
14 stops = data_sorted[(data_sorted['stop_begin']) | (data_sorted['stop_end'])]
15 stops = stops[['time', 'vehicle_id', 'stop_begin', 'stop_end']]
16 stops['end_time'] = stops['time'].where(stops['stop_end']).groupby(stops['vehicle_id']).shift(-1)
17 stops['duration'] = stops['end_time'] - stops['time']
18 return stops[['time', 'duration']].dropna()
19
20 def create_signal_array(stops, total_time=3600):
21     signal = np.zeros(total_time)
22     for _, row in stops.iterrows():
23         start = int(row['time'])
24         duration = int(row['duration'])
25         signal[start:start + duration] = 1
26     return signal
27
28 def estimate_traffic_cycles(stops, total_time=3600, sampling_rate=1):
29     signal = create_signal_array(stops, total_time)
30     yf = fft(signal)
31     xf = fftfreq(total_time, d=1/sampling_rate)
32     idx = np.argmax(np.abs(yf[1:])) + 1
33     dominant_frequency = xf[idx]
34     cycle_time = 1 / dominant_frequency
35     return cycle_time
36
37 def refine_stop_periods(stops, signal, cycle_estimate):
38     cycle_length = int(round(cycle_estimate))
39     stop_durations = np.zeros(len(signal))
40     for index, row in stops.iterrows():
41         start = int(row['time'])
42         duration = int(row['duration'])
43         stop_durations[start:start+duration] = 1
44     stop_frequencies = np.array([np.mean(stop_durations[i::cycle_length]) for i in range(cycle_length)])
45     threshold = 0.5
46     likely_traffic_stops = stop_frequencies > threshold
47     return likely_traffic_stops, stop_frequencies
48
49 def estimate_red_green_durations(traffic_stops, cycle_length):
50     red_duration = sum(traffic_stops)
51     green_duration = cycle_length - red_duration
52     return red_duration, green_duration
53
54 def process_intersection_data(filepath):

```

```

55 data = pd.read_csv(filepath)
56 stops = detect_stop_periods(data)
57 traffic_cycle = estimate_traffic_cycles(stops)
58 signal = create_signal_array(stops)
59 likely_traffic_stops, _ = refine_stop_periods(stops, signal, traffic_cycle)
60 red_duration, green_duration = estimate_red_green_durations(likely_traffic_stops,
    int(round(traffic_cycle)))
61 return red_duration, green_duration, traffic_cycle
62
63 filepaths = [
64     "C:\\Users\\13544\\Desktop\\A1.csv",
65     "C:\\Users\\13544\\Desktop\\A2.csv",
66     "C:\\Users\\13544\\Desktop\\A3.csv",
67     "C:\\Users\\13544\\Desktop\\A4.csv",
68     "C:\\Users\\13544\\Desktop\\A5.csv"
69 ]
70
71 results = {}
72 for i, filepath in enumerate(filepaths, 1):
73     intersection = f"A{i}"
74     results[intersection] = process_intersection_data(filepath)
75
76 print("Traffic light durations and cycles:")
77 for key, value in results.items():
78     print(f"{key}: Red duration = {value[0]} s, Green duration = {value[1]} s, Cycle = {value[2]} s")

```

B附录二：问题一交通信号灯随时间的变化情况

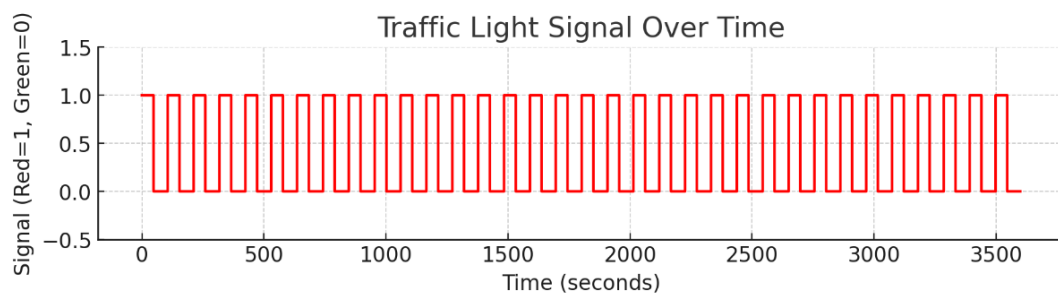


图 8 A1 路口

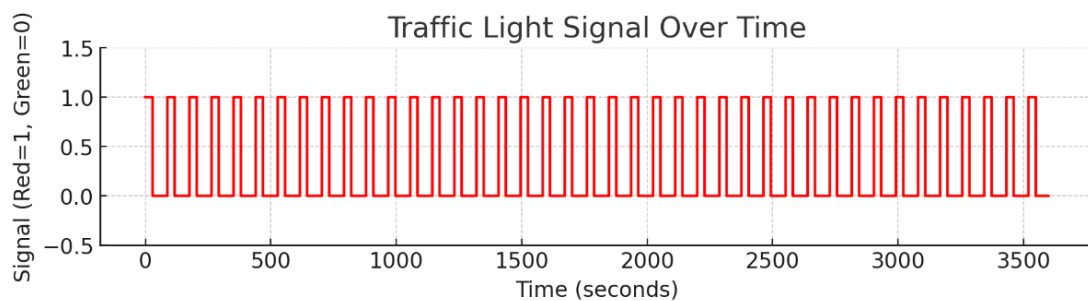


图 9 A2 路口

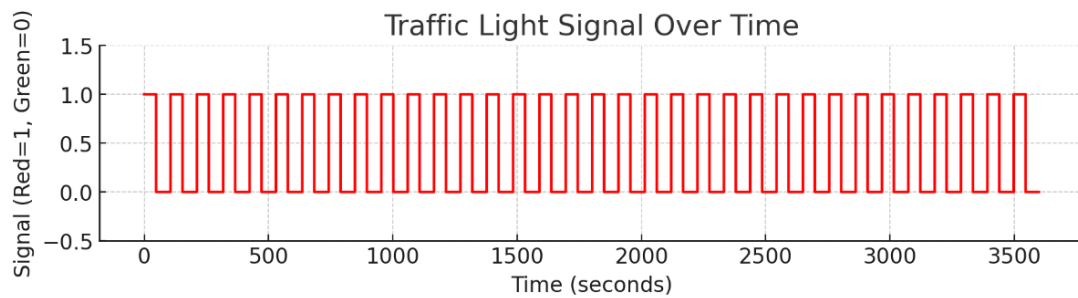


图 10 A3 路口

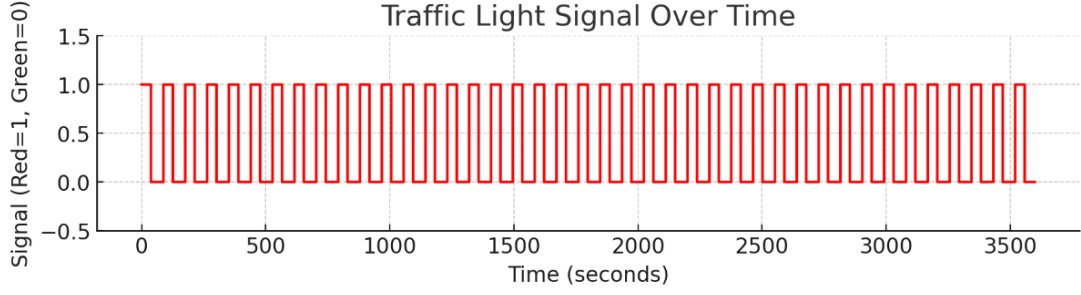


图 11 A4 路口

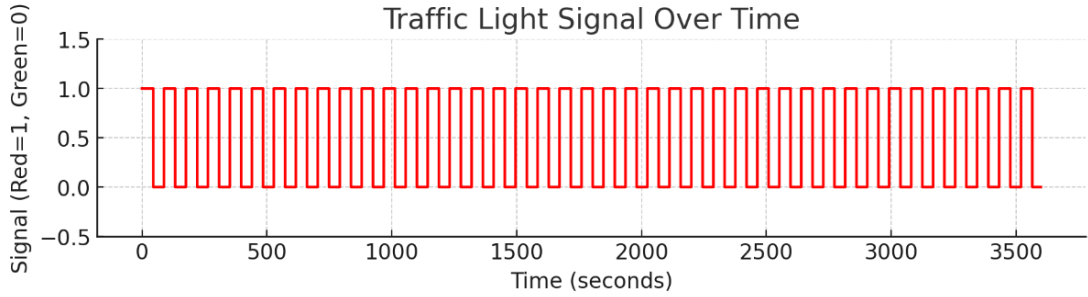


图 12 A5 路口

C附录三：问题二中对问题一模型改进后计算周期的代码

```

1 import pandas as pd
2 import numpy as np
3 from scipy.fft import fft,fftfreqimport matplotlib.pyplot as plt
4 def calculate_velocity_and_detect_stops(data, speed_threshold=0.5):
5     data_sorted = data.sort_values(by=['vehicle_id', 'time'])    data_sorted['x_diff'] =
        data_sorted.groupby('vehicle_id')['x'].diff().fillna(0)    data_sorted['y_diff'] =
        data_sorted.groupby('vehicle_id')['y'].diff().fillna(0)    data_sorted['time_diff'] =
        data_sorted.groupby('vehicle_id')['time'].diff().fillna(1)    data_sorted['speed'] =
        np.sqrt(data_sorted['x_diff']**2 + data_sorted['y_diff']**2) / data_sorted['time_diff']
        data_sorted['stopped'] = data_sorted['speed'] <= speed_threshold data_sorted['stop_begin'] =
        data_sorted['stopped'] & (~data_sorted['stopped'].shift(1).fillna(False))    data_sorted['stop_end'] =
        data_sorted['stopped'] & (~data_sorted['stopped'].shift(-1).fillna(False))    stops =
        data_sorted[(data_sorted['stop_begin']) | (data_sorted['stop_end'])]    stops = stops[['time',
        'vehicle_id', 'stop_begin', 'stop_end']]    stops['end_time'] =
        stops['time'].where(stops['stop_end']).groupby(stops['vehicle_id']).shift(-1)    stops['duration'] =
        stops['end_time'] - stops['time']    return stops[['time', 'duration']].dropna()def
        create_signal_array(stops, total_time):    signal = np.zeros(total_time)    for _, row in
        stops.iterrows():        start = int(row['time'])        duration = int(row['duration'])
        signal[start:start + duration] = 1    return signaldef refine_stop_periods(stops, signal,
        cycle_estimate):    cycle_length = int(round(cycle_estimate))    stop_durations =
        np.zeros(len(signal))    for index, row in stops.iterrows():        start = int(row['time'])
        duration = int(row['duration'])        stop_durations[start:start+duration] = 1    stop_frequencies =
        np.array([np.mean(stop_durations[i::cycle_length]) for i in range(cycle_length)])    threshold = 0.5

```

```

likely_traffic_stops = stop_frequencies > threshold return likely_traffic_stops,
stop_frequenciesdef estimate_red_green_durations(traffic_stops, cycle_length): red_duration =
sum(traffic_stops) green_duration = cycle_length - red_duration return red_duration,
green_durationdef process_intersection_data(filepath): data = pd.read_csv(filepath) stops =
calculate_velocity_and_detect_stops(data) total_time = int(stops['time'].max()) + 1 signal =
create_signal_array(stops, total_time) yf = fft(signal) xf = fftfreq(total_time, d=1) idx =
np.argmax(np.abs(yf[1:])) + 1 dominant_frequency = xf[idx] cycle_time = 1 / dominant_frequency
likely_traffic_stops, __ = refine_stop_periods(stops, signal, cycle_time) red_duration,
green_duration = estimate_red_green_durations(likely_traffic_stops, int(round(cycle_time))) return
red_duration, green_duration, cycle_timefilepaths = [ "C:\\Users\\13544\\Desktop\\B1.csv",
"C:\\Users\\13544\\Desktop\\B2.csv", "C:\\Users\\13544\\Desktop\\B3.csv",
"C:\\Users\\13544\\Desktop\\B4.csv", "C:\\Users\\13544\\Desktop\\B5.csv"]results = {}for
filepath in filepaths: intersection = filepath.split("\\")[-1].split(".")[0] results[intersection] =
process_intersection_data(filepath)print("Traffic light durations and cycles:")for key, value in
results.items(): print(f"{key}: Red duration = {value[0]} s, Green duration = {value[1]} s, Cycle
= {value[2]} s")

```

D附录四：A1 至 A5 路口车辆轨迹图

